

SIMATS Engineering — Assessment Tool 1

Course: Artificial Intelligence (CSA17) | CO1: Intelligent Agents & Uninformed Search
Answer Key — Annexure A: Analytical Problem Solving

Q1. Water Jug Problem (4-gallon & 3-gallon jugs → exactly 2 gallons in the 4-gallon jug)

Problem Formulation

State representation: (x, y) where x = amount of water in the 4-gallon jug (0–4) and y = amount of water in the 3-gallon jug (0–3).

Initial state: $(0, 0)$ — both jugs empty.

Goal state: $(2, y)$ for any y — the 4-gallon jug contains exactly 2 gallons.

Operators (production rules):

- Fill the 4-gallon jug from the pump: $(x, y) \rightarrow (4, y)$
- Fill the 3-gallon jug from the pump: $(x, y) \rightarrow (x, 3)$
- Empty the 4-gallon jug onto the ground: $(x, y) \rightarrow (0, y)$
- Empty the 3-gallon jug onto the ground: $(x, y) \rightarrow (x, 0)$
- Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full or the 3-gallon jug is empty
- Pour water from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is full or the 4-gallon jug is empty

Solution Path

Step	Action	4-gallon jug	3-gallon jug
0	Initial state	0	0
1	Fill the 3-gallon jug	0	3
2	Pour 3-gallon jug into 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour from 3-gallon jug into 4-gallon jug until 4-gallon jug is full	4	2
5	Empty the 4-gallon jug	0	2
6	Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug	2	0

At step 6 the state is $(2, 0)$ — the 4-gallon jug holds exactly 2 gallons, satisfying the goal. This path corresponds to a sequence of nodes expanded by an uninformed search strategy (e.g. BFS/DFS on the state-space graph of jug configurations), where each edge is one of the six operators above.

Q2. Mars Rover — Agent Analysis

i. Percepts

Camera/stereo-vision images of terrain, spectrometer and chemical-sensor readings from rock/soil samples, proximity and obstacle-distance readings, wheel-odometry and IMU data (position/tilt), temperature and atmospheric sensor data, battery/power level, and command signals received from Earth mission control.

ii. Environment (PEAS/environment properties)

- **Partially observable** — onboard sensors reveal only the local terrain, not the whole planet.
- **Stochastic** — dust storms, wheel slippage, and sensor noise make outcomes uncertain.
- **Sequential** — the current action (e.g. drilling, moving) affects future states and options.
- **Dynamic** — weather, lighting, and terrain can change while the rover is deliberating.
- **Continuous** — position, sensor readings, and time are continuous-valued.
- **Single-agent** (from the rover's viewpoint), though it may coordinate with orbiters/other rovers.
- **Unknown environment** initially — the map of Mars terrain is not known in advance and is built as the rover explores.

iii. Actions

Move forward/backward, turn/steer, climb over or avoid an obstacle, extend/operate the robotic arm, drill into rock or scoop soil, place a sample in an onboard instrument for analysis, capture images, transmit collected data to Earth, and enter a low-power/safe mode when needed.

iv. Performance Measure

Distance safely traversed, number and scientific value of samples collected and analysed, area of terrain mapped, accuracy/quality of transmitted data, energy efficiency (battery usage vs. task output), and avoidance of damage to the rover or loss of communication.

v. Suitable Agent Architecture

A **model-based, goal-based (and ideally utility-based) agent** is most suitable. Because the environment is partially observable and dynamic, the rover must maintain an internal model of the world (terrain map, its own position, known hazards) that is updated as new percepts arrive — a simple reflex agent cannot do this. Its behaviour is also driven by long-term mission goals (reach a target site, collect specific samples), which requires goal-based planning; and where several actions satisfy a goal (e.g. choosing which nearby rock to sample first), a utility function over cost, time, risk, and scientific value helps it choose the best one.

Q3. 8-Queens Problem — Problem Formulation & Search

Problem Components

- **States:** any arrangement of 0 to 8 queens on the chessboard, one queen at most per column.
- **Initial state:** an empty 8×8 board (no queens placed).
- **Actions/Successor function:** add a queen to the left-most empty column, in any of the rows of that column that is not attacked by an already-placed queen.
- **Goal test:** 8 queens are placed on the board such that no queen attacks another (no two queens share a row, column, or diagonal).
- **Path cost:** not relevant here — every step costs the same, so only the final configuration matters (a pure constraint-satisfaction / search-to-a-goal problem, not an optimisation one).

Why the naive formulation is bad — and the improved one

Placing any queen on any of the 64 squares at each of 8 steps gives a search space of about $64 \times 63 \times \dots \times 57 \approx 1.8 \times 10^{14}$ states. Restricting each new queen to the left-most empty column reduces this to $8^8 \approx 16.8$ million states. Further restricting placement to rows/diagonals not already attacked (incremental formulation) prunes this dramatically, leaving only 2,057 states to reach a solution — this is exactly why search strategy matters, not just the state definition.

Search Strategy

The 8-Queens problem is solved with an **uninformed, depth-first backtracking search** over the incremental formulation: place a queen in the left-most empty column in the first row that is not under attack; if a column has no safe row, backtrack to the previous column and try the next available row there. This explores the search tree depth-first and prunes branches as soon as a constraint (row/diagonal clash) is violated, so most of the raw 8^8 state space is never actually visited.

```
function BACKTRACK(board, col):
  if col == 8: return board          # goal reached
  for row in 0..7:
    if SAFE(board, row, col):
      place queen at (row, col)
      result = BACKTRACK(board, col + 1)
      if result != failure: return result
      remove queen at (row, col)    # backtrack
  return failure
```

Q4. OLA Cab Booking — Problem-Solving Agent

Type of Agent

This is best modelled as a **utility-based, goal-based simple problem-solving agent**. The goal (reach the destination) can be satisfied by several possible actions — booking a mini, micro, sedan, shared, or prime cab — so simply reaching the goal is not enough; the agent must also compare options on cost, waiting time, and comfort and pick the one with the highest utility for the customer's stated preference. This fits the classic SIMPLE-PROBLEM-SOLVING-AGENT template (formulate goal → formulate problem → search → act), extended with a utility function over the discovered options.

Pseudocode

```
function OLA-CAB-AGENT(percept) returns action
  persistent: state, goal, problem, plan

  state <- UPDATE-STATE(state, percept)  # pickup, destination, live traffic
  if plan is empty:
    goal <- FORMULATE-GOAL(destination)
    problem <- FORMULATE-PROBLEM(state, goal)
    # states = possible routes / cab-type choices
    # actions = select cab type, take route segment
    options <- SEARCH(problem)           # candidate (cab type, route) pairs
    best <- EVALUATE-UTILITY(options, preference)
    # utility = f(fare, ETA, comfort, availability)
    plan <- SEQUENCE(best)               # e.g. [book(best), navigate(route)]

  action <- FIRST(plan)
  plan <- REST(plan)
  return action

function EVALUATE-UTILITY(options, preference) returns best option
  for each option in options:
    option.utility <- w1*(1/fare) + w2*(1/ETA) + w3*comfort_score
  return option with max utility
```

Q5. Least-Cost Delivery Route — Uniform Cost Search (UCS)

Note: the weighted delivery-network diagram referenced in the question was not included in the uploaded file, so the algorithm is explained in full generality below and demonstrated on a representative sample graph. Substitute your assignment's actual warehouse nodes and edge costs into the same procedure to get the numeric least-cost path from S to G.

How UCS Works

Uniform Cost Search is an uninformed search strategy that always expands the node with the **lowest cumulative path cost $g(n)$** from the start node, rather than the shallowest node (as in BFS). It uses a priority queue ordered by $g(n)$ and is guaranteed to find the optimal (least-cost) path provided all edge costs are non-negative.

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node <- root node with state = problem.INITIAL-STATE, g = 0
  frontier <- priority queue ordered by g, initially containing node
  explored <- empty set
  loop:
    if frontier is empty: return failure
    node <- POP(frontier)           # lowest g(n) so far
    if problem.GOAL-TEST(node.state): return SOLUTION(node)
    add node.state to explored
    for each action in problem.ACTIONS(node.state):
      child <- CHILD-NODE(problem, node, action)
      if child.state not in explored and not in frontier:
        add child to frontier
      else if child.state in frontier with higher g:
        replace that frontier node with child  # found cheaper path
```

Worked Example on a Representative Warehouse Network

Assume warehouses S (start), A, B, C, and G (goal) with the following routes and travel costs (fuel + time): S→A = 2, S→B = 5, A→B = 1, A→C = 8, B→C = 2, C→G = 3, B→G = 9.

Step	Node Expanded	$g(n)$	Frontier (state : g)
1	S	0	A:2, B:5
2	A	2	B:3 (via A, cheaper than 5), C:10 (via A)
3	B	3	C:5 (via B, cheaper than 10), G:12 (via B)
4	C	5	G:8 (via C, cheaper than 12)
5	G (goal)	8	— search stops, optimal path found

Result: the least-cost path is $S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$ with total cost 8 ($2 + 1 + 2 + 3$), which UCS guarantees is optimal because it always expands the cheapest frontier node first and updates a node's cost whenever a cheaper path to it is discovered.